TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TLAXIACO

FUNDAMENTOS DE INGENIERIA DE SOFTWARE

Diagrama de clases (UML) del módulo del Hub

PRESENTA:
NELSON MACARIO MARTÍNEZ HERNÁNDEZ
LEWIS NOE HERNÁNDEZ MENDOZA
LUIS ÁNGEL GARCÍA SANTIAGO

GRUPO:
5BS

DOCENTE:
ROMAN CRUZ JOSE ALFREDO

CARRERA:
ING. EN SISTEMAS COMPUTACIONALES

Tlaxiaco, Oax., 10 de noviembre de 2026

Contenido

1. Introducción
2. Identificación de clases desde el DRS
3. Diagrama de clases
4. Enumeraciones
5. Especificación de clases
6. Asociaciones y multiplicidad
7. Encapsulamiento y visibilidad
8. Agregación, composición y herencia
9. Trazabilidad con el DRS y el BPMN
10. Notación UML y uso de Architect
11. Trabajo por pull request y revisión cruzada
12. Verificación contra la rúbrica
13. Conclusión

1. Introducción

Este documento presenta el modelo de análisis del módulo MezcalTrace, parte de Open Hub Tec, expresado como un diagrama de clases UML. Open Hub Tec es el proyecto integrador del Instituto Tecnológico de Tlaxiaco, y MezcalTrace es la propuesta de solución para Tu Tienda Artesanal, una organización que comercializa y distribuye mezcal artesanal en Tlaxiaco, Oaxaca. El mezcal que manejan viene de San Carlos Yautepec, en el Istmo de Tehuantepec.

La idea del módulo es sencilla: que cada lote de mezcal tenga un código QR y que cualquier comprador pueda escanearlo para saber de dónde viene y por quién pasó antes de llegar a sus manos. Esa necesidad salió de la entrevista con el comercializador de la organización en el Sprint 1 y quedó recogida en los requisitos del Documento de Especificación de Requisitos de Software (DRS) validado en la Unidad 3.

El propósito del modelo es traducir esos requisitos a una representación orientada a objetos que sirva de base para el diseño. Para construirlo se aplicó la carrera de sustantivos sobre los requisitos del DRS y las relaciones entre clases se definieron con las reglas que expresó la organización. Cada elemento conserva el identificador del requisito que lo originó.

El modelo consta de seis clases, dos enumeraciones y cinco asociaciones con nombre y multiplicidad. El paquete de destino en Architect es Análisis.

2. Identificación de clases desde el DRS

Se aplicó la carrera de sustantivos sobre los requisitos funcionales del DRS. Cada sustantivo se conservó como clase, se pasó a atributo o se descartó, según el criterio que muestra la tabla.

Sustantivo del DRS	Decisión	Razón
Productor, maestro mezcalero	Clase Productor	Quien elabora el mezcal (RF-01). Se usa el vocabulario de la organización.
Lote, mezcal, producción	Clase Lote	Unidad que se rastrea y a la que se asocia el QR (RF-02). Mezcal y producción son sinónimos de lote en este contexto.
Historial, etapa, movimiento	Clase EventoLote	Cada paso del lote (elaborado, envasado, distribuido) se registra con fecha (RF-06).
Comercializador, distribuidor	Clase Comercializador	Quien recibe y vende el lote (RF-04). Distribuidor es sinónimo.
Código QR, etiqueta	Clase CodigoQR	Se genera por lote y se escanea para consultar el origen (RF-03).
Consulta, escaneo	Clase ConsultaOrigen	Cada vez que un comprador consulta un lote por QR (RF-05).
Comprador, cliente	No es clase	Es el actor que hace la consulta. No necesita registrarse, así que no guarda datos propios.
Litros, fecha, nombre, comunidad, teléfono, tipo de maguey	Atributos	Datos simples de Productor, Lote o Comercializador.
Estado del lote, medio de consulta	Enumeraciones	Valores fijos del DRS.

Sustantivo del DRS	Decisión	Razón
Rapidez, pocos pasos desde el celular	No es clase	Viene del requisito no funcional RNF-01: es una restricción y no una entidad.
Sistema, pantallas, aplicación	Descartados	No son entidades del dominio.

3. Diagrama de clases

El diagrama contiene seis clases, dos enumeraciones y una nota de trazabilidad por clase. Las asociaciones muestran nombre y multiplicidad en ambos extremos.

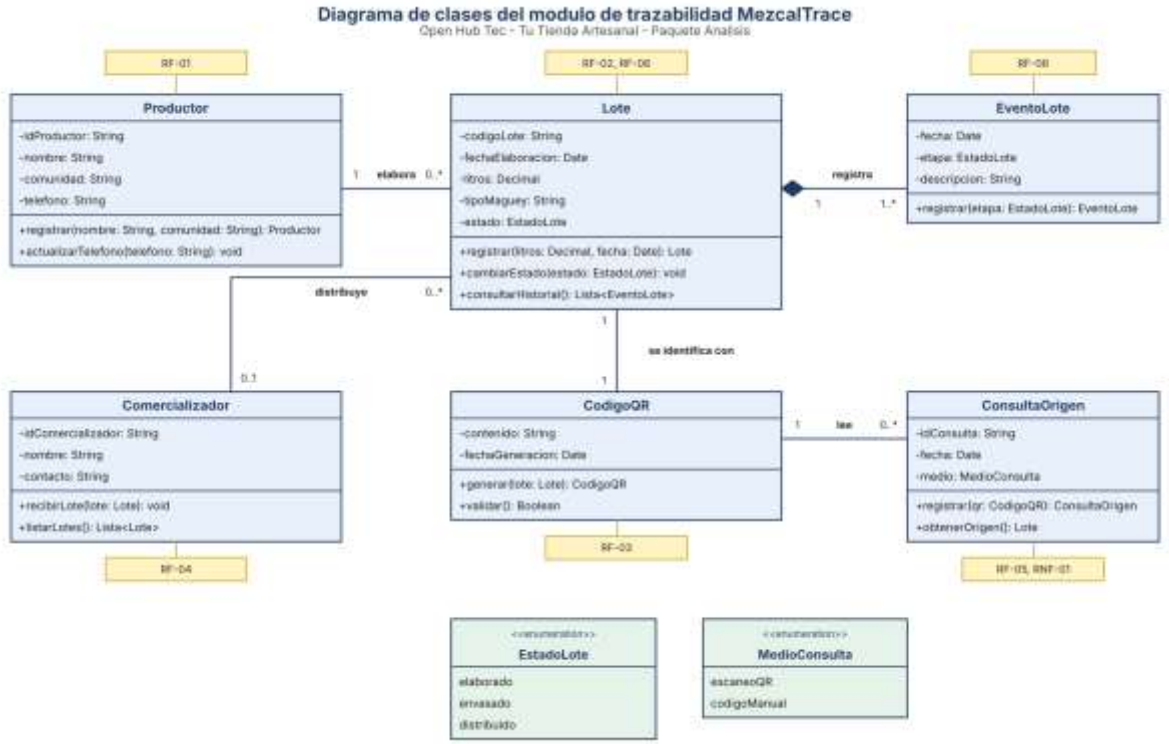


Figura 1. Diagrama de clases del módulo MezcalTrace (Open Hub Tec).

4. Enumeraciones

Los estados y medios del DRS se modelan como enumeraciones para dar un tipo coherente a los atributos que las usan.

Enumeración	Valores	Requisito
EstadoLote	elaborado, envasado, distribuido	RF-02, RF-06
MedioConsulta	escaneoQR, codigoManual	RF-05, RNF-01

5. Especificación de clases

Cada clase se describe con sus atributos (tipo y descripción) y sus operaciones (parámetros y tipo de retorno). Las operaciones salen de los verbos de los requisitos y están en la clase que tiene los datos que usan.

5.1 Productor

Requisitos: RF-01. Persona que elabora el mezcal y lo registra como suyo.

Miembro	Tipo / retorno	Descripción
Atributos		
-idProductor	String	Identificador único.
-nombre	String	Nombre del productor.
-comunidad	String	Comunidad de origen, por ejemplo San Carlos Yautepec.
-telefono	String	Teléfono de contacto.
Operaciones		
+registrar(nombre: String, comunidad: String)	Productor	Da de alta al productor (RF-01).
+actualizarTelefono(telefono: String)	void	Cambia el teléfono de contacto.

5.2 Lote

Requisitos: RF-02, RF-06. Tanda de mezcal elaborada por un productor. Es lo que se rastrea.

Miembro	Tipo / retorno	Descripción
Atributos		
-codigoLote	String	Código único del lote.
-fechaElaboracion	Date	Fecha en que se elaboró.
-litros	Decimal	Volumen producido, con decimales.
-tipoMaguey	String	Maguey con el que se elaboró.
-estado	EstadoLote	Etapa actual del lote.
Operaciones		
+registrar(litros: Decimal, fecha: Date)	Lote	Crea el lote (RF-02).
+cambiarEstado(estado: EstadoLote)	void	Avanza la etapa y deja un evento en el historial (RF-06).
+consultarHistorial()	Lista<EventoLote>	Devuelve los eventos del lote en orden (RF-06).

5.3 EventoLote

Requisitos: RF-06. Paso del historial de un lote. No existe sin su lote (composición).

Miembro	Tipo / retorno	Descripción
Atributos		
-fecha	Date	Cuándo ocurrió.

Miembro	Tipo / retorno	Descripción
-etapa	EstadoLote	Etapa que se registró.
-descripcion	String	Detalle breve del movimiento.
Operaciones		
+registrar(etapa: EstadoLote)	EventoLote	Agrega un paso al historial del lote.

5.4 Comercializador

Requisitos: RF-04. Quien recibe lotes del productor y los vende.

Miembro	Tipo / retorno	Descripción
Atributos		
-idComercializador	String	Identificador único.
-nombre	String	Nombre del comercializador.
-contacto	String	Dato de contacto.
Operaciones		
+recibirLote(lote: Lote)	void	Registra que el lote le fue entregado (RF-04).
+listarLotes()	Lista<Lote>	Muestra los lotes que distribuye.

5.5 CodigoQR

Requisitos: RF-03. Código que identifica a un lote y permite consultar su origen.

Miembro	Tipo / retorno	Descripción
Atributos		
-contenido	String	Cadena que codifica el QR.
-fechaGeneracion	Date	Cuándo se generó.
Operaciones		
+generar(lote: Lote)	CodigoQR	Crea el código del lote (RF-03).
+validar()	Boolean	Revisa que el código corresponda a un lote real.

5.6 ConsultaOrigen

Requisitos: RF-05, RNF-01. Cada vez que alguien consulta el origen de un lote por QR.

Miembro	Tipo / retorno	Descripción
Atributos		
-idConsulta	String	Identificador único.
-fecha	Date	Cuándo se consultó.
-medio	MedioConsulta	Escaneo de QR o captura manual del código.

Miembro	Tipo / retorno	Descripción
Operaciones		
+registrar(qr: CodigoQR)	ConsultaOrigen	Guarda la consulta (RF-05).
+obtenerOrigen()	Lote	Devuelve el lote con su productor e historial (RF-05).

6. Asociaciones y multiplicidad

Cada asociación tiene nombre y multiplicidad en los dos extremos. Datos como el productor de un lote o el lote de un código QR se expresan como asociaciones y no se repiten como atributos.

Clase A	Asociación	Clase B	Mult. A	Mult. B	Regla
Productor	elabora	Lote	1	0..*	Un productor elabora varios lotes; cada lote es de un solo productor (RF-01, RF-02).
Lote	registra (composición)	EventoLote	1	1..*	Todo lote tiene al menos un evento (el de elaboración); el evento no existe sin su lote (RF-06).
Lote	se identifica con	CodigoQR	1	1	Cada lote tiene un único código y cada código identifica a un solo lote (RF-03).
Comercializador	distribuye	Lote	0..1	0..*	Un lote puede no tener comercializador todavía; un comercializador maneja varios lotes (RF-04).
ConsultaOrigen	lee	CodigoQR	0..*	1	Un código se puede consultar muchas veces; cada consulta lee un código (RF-05).

Las multiplicidades 0..1 y 1..* se propusieron a partir de lo que dijo la organización y se deben confirmar con ella.

7. Encapsulamiento y visibilidad

- Todos los atributos son privados (-): el estado de un objeto solo cambia mediante sus operaciones. Por ejemplo, el estado de un lote se modifica únicamente con cambiarEstado().
- Todas las operaciones son públicas (+), porque son las que usan los casos de uso del DRS.
- No se usa visibilidad protegida (#) porque el modelo no tiene herencia.
- No hay excepciones a esta regla; la visibilidad es la misma en las seis clases.

8. Agregación, composición y herencia

- Composición Lote ◆— EventoLote. Un evento del historial no tiene sentido sin el lote al que pertenece: si el lote se elimina, su historial desaparece con él. El rombo relleno está del lado del todo (Lote).
- Sin agregación. Ninguna otra clase es un todo con partes que puedan existir por separado. Un código QR podría pensarse como parte del lote, pero se modeló como asociación 1 a 1 porque el código se genera y se valida con su propio ciclo.
- Sin herencia. Productor y Comercializador comparten solo el nombre y un dato de contacto, y se comportan distinto (uno elabora, el otro recibe). Hacerlos subclases de una clase Persona sería una herencia forzada, así que se dejaron como clases separadas.

9. Trazabilidad con el DRS y el BPMN

Cada clase del diagrama tiene una nota con los requisitos que la originan. Esta tabla es la base de trazabilidad.csv y cubre todas las clases y enumeraciones. Los nombres coinciden con los del DRS y con el BPMN mejorado de la Unidad 2.

Elemento	Tipo	Requisitos	Origen BPMN
Productor	Clase	RF-01	Registro de productor
Lote	Clase	RF-02; RF-06	Elaboración y registro de lote
EventoLote	Clase	RF-06	Historial del lote
Comercializador	Clase	RF-04	Distribución del lote
CodigoQR	Clase	RF-03	Generación de código QR
ConsultaOrigen	Clase	RF-05; RNF-01	Consulta de origen por QR
EstadoLote	Enumeración	RF-02; RF-06	Elaboración y distribución
MedioConsulta	Enumeración	RF-05; RNF-01	Consulta de origen por QR

10. Notación UML y uso de Architect

- El diagrama de clases representa el modelo de análisis del módulo MezcalTrace y está en el paquete Análisis, como diagrama de tipo Class.
- Clases y enumeraciones en UpperCamelCase (EventoLote, ConsultaOrigen); atributos y operaciones en lowerCamelCase (fechaElaboracion, cambiarEstado).
- Las clases se crean como elementos del modelo, no como figuras sueltas; las enumeraciones llevan el estereotipo enumeration.
- La Figura 1 se exporta como PNG de más de 800 px de ancho.

11. Trabajo por pull request y revisión cruzada

11.1 Flujo de trabajo

- Cada cambio se registra mediante un pull request con la plantilla llena y la aprobación de otro integrante.

- El archivo .qea se edita por turnos para evitar conflictos.
- Todos los integrantes aparecen como autores o revisores de al menos un pull request.

Integrante	Pull request como autor	Pull request como revisor
Nelson Macario Martínez Hernández		
Lewis Noe Hernández Mendoza		
Luis Ángel García Santiago		

11.2 Registro de la revisión cruzada

El equipo revisor evalúa el diagrama con la rúbrica de la práctica. Cada observación se responde en el pull request y se corrige o se justifica; aquí se anota qué cambió y por qué.

Observación del equipo revisor	Respuesta (corregido / justificado)	Qué cambió y por qué

12. Verificación contra la rúbrica

Criterio	Evidencia en este documento
1. Clases identificadas desde el DRS	Sección 2: carrera de sustantivos; 6 clases con vocabulario de la organización; sin Sistema ni atributos disfrazados.
2. Atributos y tipos de datos	Sección 5: todos los atributos con tipo (Decimal, Date, enumeraciones); ninguno repite una relación.
3. Operaciones ligadas a los requisitos	Sección 5: operaciones con parámetros y tipo de retorno, citando su RF.
4. Encapsulamiento y visibilidad	Sección 7: atributos privados (-), operaciones públicas (+), sin protegido.
5. Asociaciones y multiplicidad	Sección 6: 5 asociaciones con nombre y multiplicidad en ambos extremos.
6. Agregación, composición y herencia	Sección 8: una composición justificada; sin agregación ni herencia, con justificación.
7. Trazabilidad con el DRS y el BPMN	Sección 9, trazabilidad.csv y notas de requisitos de la Figura 1.
8. Notación UML y uso de Architect	Secciones 3 y 10.
9. Trabajo por pull request	Sección 11.1.
10. Revisión cruzada y mejora	Sección 11.2.

13. Conclusión

El diagrama de clases permitió pasar de los requisitos del DRS a un modelo concreto del módulo MezcalTrace. A partir de la carrera de sustantivos salieron seis clases (Productor, Lote, EventoLote, Comercializador, CodigoQR y ConsultaOrigen) y dos enumeraciones, cada una con atributos con tipo, operaciones ligadas a un requisito y una nota que cita el RF o RNF que la originó.

Hubo decisiones de modelado que conviene destacar. El historial del lote se hizo clase aparte y se ligó con composición, porque un paso del historial no existe sin su lote. Productor y Comercializador se dejaron separados en lugar de forzar una herencia. Y el comprador no se modeló como clase, porque solo consulta y no guarda datos.

Armar el modelo también mostró lo útil que es la trazabilidad: tener el identificador del requisito en cada clase ayuda a ver qué se afecta si algo cambia, y deja el camino listo para la etapa de diseño.